

The Decision Tree Learning Algorithm 🌳

The process of building a decision tree is **recursive**. It starts with the entire dataset at the root node and iteratively splits the data into smaller, purer subsets until a stopping criterion is met.

The Core Algorithm (Iterative Splitting)

1. **Start at the Root Node:** Begin with the complete set of training examples.
2. **Check Stopping Criteria:** At the current node, first check if any **stopping criteria** (see below) have been met.
 - **If YES:** Stop splitting. This node becomes a **leaf node**. Assign it the majority class of the examples it contains (e.g., "Cat" if 4 out of 5 examples are cats).
 - **If NO:** Proceed to step 3.
3. **Find the Best Split:**
 - Calculate the **information gain** for *every* possible feature on the *current* subset of data.
 - Select the feature that provides the **highest information gain**.
4. **Create Branches:**
 - Split the current node's dataset into two new subsets (e.g., a left branch and a right branch) based on the values of the feature you just selected.
5. **Recurse:**
 - Apply this entire process (starting from Step 2) independently to both the left and right branches.

This recursive process (a function that calls itself on smaller subsets) continues until all branches have ended in leaf nodes.

Stopping Criteria (Preventing Overfitting)

If the algorithm continued splitting until every node was 100% pure, the tree would become massive, complex, and would **overfit** the training data. To prevent this, we use **stopping criteria** (also called **pre-pruning**):

- **1. Maximum Purity:** Stop splitting if the node is already 100% pure (i.e., all examples belong to a single class; entropy is 0).
- **2. Maximum Depth:** Stop splitting if the tree reaches a predefined `max_depth` (a hyperparameter, e.g., 10 levels deep).
- **3. Minimum Information Gain:** Stop if the best possible split provides an information gain that is below a small threshold (i.e., the split isn't "worth it").

- **4. Minimum Examples in Node:** Stop if the number of examples in the current node falls below a certain threshold.
-

Hyperparameters and Overfitting

The `max_depth` parameter is a key tool for controlling the bias-variance trade-off in a decision tree:

- **A large `max_depth`** (or no limit) allows the tree to grow very large and complex. It can fit the training data perfectly but is at high risk of **overfitting (high variance)**.
- **A small `max_depth`** forces the tree to be simple and shallow. It is less likely to overfit but is at high risk of **underfitting (high bias)**.

You can use a cross-validation set to find the optimal value for hyperparameters like `max_depth`.

Making Predictions (Inference)

Once the tree is built, making a prediction on a new example is simple:

1. Start at the root node.
2. Follow the branches down the tree based on the example's feature values.
3. The prediction is the class specified by the leaf node you arrive at.